Introduction to Python and its Features

- Python is a high-level, interpreted, and general-purpose programming language created by Guido van Rossum in 1991. It is easy to learn because of its simple and readable syntax.

Main Features of Python:

- Simple and Easy to Learn – Python syntax is easy to read and write.
- High-Level Language – No need to manage memory or hardware details.
- Interpreted Language – Code runs line by line using an interpreter.
- Platform Independent – Python programs run on Windows, Linux, and macOS.
- Large Library Support – Provides many built-in libraries and modules.
- Object-Oriented – Supports classes, objects, and other OOP concepts.
- Open Source – Free to use and supported by the Python Software Foundation.

History and Evolution of Python

- Python was created by Guido van Rossum and first released in 1991. It was developed to make programming simple, readable, and easy to learn.
- Python was influenced by programming languages like ABC, C,Over time, many versions of Python were released with improved features.

Major Versions:

- Python 1.0 (1991): First version with basic programming features.
- Python 2.0 (2000): Added features like list comprehensions and garbage collection.
- Python 3.0 (2008): Major update with improved syntax and better performance.

- Today, Python is maintained by the Python Software Foundation and is widely used in web development, data science, AI, and automation.

Advantages of Using Python Over Other Programming Languages

1. Easy to Learn and Use – Python has simple and readable syntax, making it beginner-friendly.
2. Less Code – Python requires fewer lines of code compared to many other languages.
3. Interpreted Language – Code runs line by line, making debugging easier.
4. Large Standard Library – Provides many built-in modules for different tasks.
5. Platform Independent – Python programs run on Windows, Linux, and macOS.
6. Supports Multiple Programming Styles – Supports procedural, object-oriented, and functional programming.
7. Strong Community Support – A large developer community provides tutorials, tools, and support.
8. Open Source – Free to use and maintained by the Python Software Foundation.

Installing Python and Setting Up the Development Environment

1. **Install Python**
   Download Python from the official website and install it on your system.
2. **Using Anaconda**

- Download and install Anaconda.
- It includes Python and many useful libraries for data science.
- Use Anaconda Navigator or Jupyter Notebook to write Python code.

3. **Using** PyCharm

- Install PyCharm IDE.
- Create a new Python project.
- Write and run Python programs easily with built-in tools.

4. **Using** Visual Studio Code

- Install VS Code.
- Install the Python extension.
- Create a .py file and run Python programs directly in the editor.

Writing and Executing Your First Python Program

The first program usually written in any programming language is the Hello World program.

Step 1: Write the Program

Open any editor like **Visual Studio Code**, **PyCharm**, or **IDLE** and write the following code:

print("Hello, World!")

Step 2: Save the File

Save the file with the .py extension.
Example: hello.py

Step 3: Execute the Program

Run the program using the Python interpreter.

Understanding Python's PEP 8 guidelines

is the official **style guide for writing clean and readable Python code**. It provides rules and recommendations so that Python programs look consistent and easy to understand.

Main PEP 8 Guidelines

1. **Indentation**
   Use **4 spaces** for indentation.

2. **Line Length**
   Each line should not exceed **79 characters**.

3. **Naming Conventions**

   - Variables and functions: `lowercase_with_underscores`
   - Classes: `CapitalizedWords`

4. **Spaces in Code**

   - No Spacing around Operator

Indentation, Comments, and Naming Conventions in Python

1. Indentation

Indentation means adding spaces at the beginning of a line to define a block of code.Python uses **4 spaces** for indentation.

2. Comments

Comments are used to explain the code and make it easier to understand. Python ignores comments during execution.

3. Naming Conventions

Naming conventions are rules for naming variables, functions, and classes.

Examples:

- **Variables:** student_name, total_marks
- **Functions:** calculate_sum()
- **Classes:** StudentData

Writing Readable and Maintainable Code

Readable and maintainable code means writing programs that are **easy to understand, modify, and debug**.

Key Practices

1. **Use Meaningful Variable Names**
   Choose clear names that describe the purpose.
   Example: total_marks instead of tm.
2. **Follow Proper Indentation**
   Use 4 spaces for indentation to keep code structured.
3. **Write Comments**
   Add comments to explain important parts of the code.
4. **Keep Code Simple**
   Avoid complex logic and write clear statements.
5. **Follow Python Style Guidelines**
   Follow **PEP 8** to maintain consistent coding style.

Understanding data types: integers, floats, strings, lists, tuples, dictionaries, sets

1. Integer (int)

- An **integer** represents whole numbers without a decimal point.
- It can be positive, negative, or zero.

2. Float (float)

- A float represents numbers that contain a decimal point.
- It is used to store real or fractional numbers.

3. String (str)

- A **string** is a sequence of characters used to represent text.
- Strings are written inside single quotes or double quotes.

4. List

- A list is an ordered collection of elements.
- It is **mutable**, meaning its elements can be modified after creation.
- Lists allow duplicate values.

5. Tuple

- A tuple is an ordered collection of elements similar to a list.
- However, it is **immutable**, meaning its elements cannot be changed after creation.

6. Dictionary

- A **dictionary** is a collection of data stored in **key–value pairs**.
- Each key is unique and is used to access its corresponding value.

7. Set

- A **set** is an unordered collection of unique elements.
- It does not allow duplicate values.

Python Variables and Memory Allocation

- Python variables are used to store data values. Unlike many other programming languages, Python does not require explicit declaration of a variable type. The type of a variable is determined automatically based on the value assigned to it. This feature is known as dynamic typing.

- A variable in Python is simply a name that refers to an object stored in memory. When a value is assigned to a variable, Python creates an object in memory and the variable becomes a reference to that object.

- For example, when a value is assigned to a variable, Python allocates memory for that value and stores it. The variable then points to the memory location where the object is stored.

Python Operators

1. Arithmetic Operators

Arithmetic operators are used to perform **mathematical calculations** such as addition, subtraction, multiplication, and division.

2. Comparison Operators

Comparison operators are used to **compare two values**. The result of the comparison is always **True or False**.

3. Logical Operators

Logical operators are used to **combine multiple conditions** and return a boolean result.

4. Bitwise Operators

Bitwise operators are used to perform operations on **binary numbers (bits)**. They work at the bit level of integers.

Introduction to Conditional Statements in Python

1. if Statement

The if statement is used to test a condition. If the condition is true, the block of code inside the if statement is executed.

2. else Statement

The else statement is used with an if statement. It executes a block of code when the condition in the if statement is false.

3. elif Statement

The elif (else if) statement is used to check multiple conditions. If the first condition is false, the program checks the next condition using elif. This allows multiple alternative conditions to be evaluated.

Nested if-else conditions

- A nested if–else condition means placing an if–else statement inside another if or else block. It is used when a program needs to check multiple conditions in a step-by-step manner.
- In nested conditions, the first if statement checks the main condition. If it is true, another if–else statement inside it checks additional conditions. Similarly, nested conditions can also appear inside the else block.

Introduction to for and while Loops

Loops in Python are used to execute a block of code **repeatedly** until a certain condition is met. They help reduce repetition and make programs more efficient.

1. for Loop

A **for loop** is used to iterate over a sequence such as a list, tuple, string, or range of numbers. It is commonly used when the number of iterations is known.

2. while Loop

A **while loop** is used to execute a block of code as long as a specified condition remains **true**. The loop continues running until the condition becomes **false**.

How Loops Work in Python

In Python, loops work by repeatedly executing a block of code based on a condition or sequence.

- In a **for loop**, the loop runs once for each item in a sequence.
- In a **while loop**, the loop continues to execute as long as the condition is true.

Using Loops with Collections (Lists, Tuples, etc.)

- In Python, **loops** are commonly used to iterate through **collections** such as lists, tuples, sets, and dictionaries. A collection is a group of multiple items stored in a single variable.
- When loops are used with collections, the loop processes **each element of the collection one by one**. This allows the program to perform operations on all items without writing separate code for each element.
- The **for loop** is most commonly used with collections because it automatically goes through each item in the collection in sequence.

Defining and Calling Functions in Python

A **function** in Python is a block of organized and reusable code used to perform a specific task. Functions help make programs more structured, readable, and reusable.

Defining a Function:

- In Python, a function is defined using the **def keyword**, followed by the function name and parentheses. The code that performs the task is written inside the function block.

Calling a Function:

- After a function is defined, it must be **called** to execute the code inside it. Calling a function means writing the **function name followed by parentheses**.
- Functions help divide a large program into smaller parts, making the code easier to understand, maintain, and reuse.

Built-in Methods for Strings, Lists, etc.

Python provides many built-in methods that allow programmers to perform common operations on data types such as strings, lists, tuples, dictionaries, and sets. These methods are predefined functions that are associated with specific data types and help in manipulating or processing data easily.

String Methods:

String methods are used to perform operations on text data, such as converting text to uppercase or lowercase, removing spaces, replacing characters, and searching for specific text within a string.

List Methods:

List methods allow modification and management of list elements. They can be used to add new items, remove items, sort elements, or change the order of elements in a list.

Tuple Methods:

Since tuples are immutable, they have very limited methods. The available methods are mainly used to count elements or find the position of an element.

Dictionary Methods:

Dictionary methods help manage key–value pairs. They allow accessing keys, values, and items, as well as adding, updating, or removing entries.

Set Methods:

Set methods are used to perform operations on sets such as adding elements, removing elements, and performing mathematical set operations like union, intersection, and difference.

Understanding the Role of break, continue, and pass in Python Loops

In Python loops, **break, continue, and pass** are control statements used to change the normal flow of loop execution.

break

The **break** statement is used to **immediately terminate the loop**. When the break statement is encountered, the loop stops running and the program continues with the next statement after the loop.

continue

The **continue** statement is used to **skip the current iteration of the loop** and move to the next iteration. The remaining code inside the loop for that iteration is not executed.

pass

The **pass** statement is a **null statement**, meaning it does nothing. It is used as a placeholder when a statement is required syntactically but no action needs to be performed.

Basic Operations on Strings

Strings in Python support several **basic operations** that allow manipulation and processing of text data.

Concatenation

Concatenation is the process of **joining two or more strings together** to form a single string.

Repetition

Repetition is used to **repeat a string multiple times**. It allows the same text to appear several times in a single string.

String Methods:

Python provides many **built-in string methods** to perform operations on strings. Some common methods include:

- **upper()** – Converts all characters of a string to uppercase.
- **lower()** – Converts all characters of a string to lowercase.
- **capitalize()** – Converts the first character of the string to uppercase.
- **strip()** – Removes leading and trailing spaces from a string.
- **replace()** – Replaces one part of a string with another.
- **find()** – Searches for a substring and returns its position.

These methods make it easier to modify and analyze text data.

String Slicing

String slicing is used to **extract a portion of a string**. It allows access to a specific range of characters within the string.

In slicing, a starting position and an ending position are used to select a part of the string. The starting index is included, while the ending index is not included.